fplot Wiki Documentation

Contents

1 Welcome to the fplot Wiki!

fplot is a plotting library for Fennel and Lua that makes it easy to create a wide range of plots using Gnuplot. This wiki provides comprehensive documentation to help you get started making the most of its features.

Getting Started

If you're new to `fplot`, here are a few pages to get you up and running:

- **Installation:** Learn how to install `fplot` and its dependencies.
- **Getting Started Tutorial:** A step-by-step guide to creating your first plot.

Core Concepts

Understanding these two concepts is key to using `fplot` effectively:

- **Configuration Options:** This is where you define the overall appearance of your plot, such as its title, labels, size, and output format.
- **Dataset Configuration:** This is where you define the data to be plotted, along with its style, title, and other attributes.

Advanced Topics

Once you've mastered the basics, you can explore more advanced features:

- **Advanced Features:** Learn about creating multiplots, adding custom labels and arrows, and more.

    Macros: A dedicated page for creating various macros

- **Examples:** A gallery of examples showcasing various types of plots you can create with `fplot`.
- **BioTech:** A gallery of examples showcasing biotech relates plots.

Why fplot?

`fplot` makes `gnuplot` a programmatic citizen in your Fennel and Lua projects. Instead of shelling out to separate scripts, you can define, generate, and integrate plots directly within your application's (and/or pipeline's) logic. `fplot` replaces the error-prone process of building command strings with a clean, table-based API where you describe your plot as a data structure. This makes your plotting code more readable, but more importantly, it makes your plots composable: you can build, combine, and reuse styles and datasets as regular pieces of data in your code.

Why Fennel?

`fplot` is written in Fennel and is designed to feel most natural when used from a `.fnl` file. While it has first-class support for Lua, here are a few reasons why you might enjoy using Fennel for your plotting scripts:

- **Concise Syntax for Data:** Fennel's Lisp-based syntax is exceptionally clean for defining the nested tables that fplot relies on. This often results in less visual noise (fewer commas, brackets, and braces) and makes complex configurations easier to read at a glance.
- **Powerful Macros:** Fennel offers a true, Lisp-style macro system, allowing you to extend the language with new syntax and create powerful abstractions that are impossible in pure Lua. While not necessary for basic plotting, it opens the door for creating highly customized plotting DSLs (Domain-Specific Languages) tailored to your specific needs.
- **Seamless Lua Interoperability:** Fennel compiles directly to Lua. This means you lose none of Lua's performance or its rich ecosystem. You can use any Lua library (including LuaJIT) and get the best of both worlds: a powerful Lisp syntax on a fast, proven runtime.
- **A More Ergonomic API:** The fplot API was designed with Fennel's keywords (e.g., `:title`, `:x-label`) in mind. While perfectly usable from Lua with the `["key-name"]` syntax, the original `kebab-case` keywords feel most at home in a Fennel environment.

Resources

Fennel Language. (n.d.). *Official documentation*. https://fennel-lang.org/
Gnuplot. (n.d.). *Official documentation*. http://gnuplot.info/documentation.html

2 Installation

This page guides you through installing `fplot` and its required dependencies.

2.1 1. Install gnuplot

`fplot` is a high-level interface for `gnuplot`, so you must have `gnuplot` installed on your system.

macOS

Using Homebrew:

```
brew install gnuplot
```

Debian / Ubuntu

```
sudo apt-get update
sudo apt-get install gnuplot
```

Arch Linux (&MSYS2)

```
sudo pacman -S gnuplot
```

Windows

You can download a `gnuplot` installer from the official gnuplot website. After installation, ensure that the `gnuplot` executable is in your system's `PATH`.

To verify the installation, open a terminal and run:

```
gnuplot --version
```

You should see the installed version of `gnuplot`.

2.2 2. Install fplot

You can use LuaRocks, *(the package manager for Lua modules)*, to install `fplot`, or you can `git clone` the github repository.

```
luarocks install fplot
```

2.3 3. Verify the Installation

You can verify that `fplot` is installed correctly by running the following code.

Fennel

Create a file `test.fnl`:

```
(local fplot (require :fplot))
(print "fplot loaded successfully!")

(fplot.plot
{:options {:title "Test Plot"}
         :datasets [{:data [1 2 3 4]}]})
```

Run it from your terminal:

```
fennel test.fnl
```

A plot window should appear with a simple line graph.

Lua

Create a file `test.lua`:

```
local fennel = require("fennel") fennel.install()
local fplot = require("fplot")
print("fplot loaded successfully!")

fplot.plot({
        options = { title = "Test Plot" },
        datasets = {
                { data = {1, 2, 3, 4} }
        }
})
```

Run it from your terminal:

```
lua test.lua
```

A plot window should appear, just like the Fennel example.
If the plot window appears, you have successfully installed and configured `fplot`!

3 Getting Started Tutorial

This tutorial will walk you through the basics of creating a plot with fplot. We'll create a simple chart with two datasets and customize its appearance.

3.1 The Core Idea

A plot in fplot is defined by a single configuration table passed to either the `fplot.plot` (for 2D) or `fplot.splot` (for 3D) function. This table has two main keys:

- `:options`: A table that controls the overall appearance of the plot (title, labels, etc.).
- `:datasets`: A list of tables, where each table defines a set of data to be plotted.

Before running any examples, make sure you have followed the setup instructions in the **README**. The examples below assume `fplot.fnl` is in your path. For Lua examples, they assume you have `fennel.lua` and are calling `require("fennel").install()` first.

3.2 Step 1: Your First Plot

Let's start with the simplest possible plot.

Fennel

```
(local fplot (require :fplot))

(fplot.plot
{:options {:title "My First Plot"}
        :datasets [{:data [1 5 3 8 6]}]})
```

Lua

```
-- First, install the fennel loader
require("fennel").install()
local fplot = require("fplot")

fplot.plot({
        options = { title = "My First Plot" },
        datasets = {
                { data = {1, 5, 3, 8, 6} }
        }
})
```

When you run this code, fplot will open an interactive window displaying a line chart. The y-values are `1, 5, 3, 8, 6`, and the x-values default to their indices (`1, 2, 3, 4, 5`).

3.3 Step 2: Working with X and Y Coordinates

Most of the time, you'll want to specify both X and Y coordinates. You can do this by providing a list of pairs (or small tables) in the `:data` field. Let's plot the function $y = x^2$.

Fennel

```
(local fplot (require :fplot))

(local squared-data [])
(for [x -5 5]
(table.insert squared-data [x (* x x)]))

(fplot.plot
{:options {:title "Parabola"
                :x-label "x"
                :y-label "y = x^2"
                :grid true}
        :datasets [{:title "x^2"
                :style "points"
                :data squared-data}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

local squared_data = {}
for x = -5, 5 do
table.insert(squared_data, {x, x*x})
end

fplot.plot({
        options = {
                title = "Parabola",
                -- IMPORTANT: Keys with hyphens must be quoted in Lua
                ["x-label"] = "x",
                ["y-label"] = "y = x^2",
                grid = true
        },
        datasets = {
                {
                        title = "x^2",
                        style = "points",
                        data = squared_data
                }
        }
})
```

In this example, we:

- Generated data points for $y = x^2$ from $x = -5$ to 5.
- Set a title for the plot and labels for the axes in the `:options` table.
- Enabled a grid by setting `:grid true`.
- In the dataset, we gave it a `:title` to be shown in the plot's key and changed the `:style` to `"points"`.

3.4 Step 3: Plotting Multiple Datasets

To plot multiple datasets, simply add more tables to the `:datasets` list. Let's add $y = x^3$ to our previous plot.

Fennel

```
(local fplot (require :fplot))

;; Generate data
(local squared-data [])
(local cubed-data [])
(for [x -5 5 0.5]
(table.insert squared-data [x (* x x)])
(table.insert cubed-data [x (* x x x)]))

(fplot.plot
{:options {:title "Polynomials"
                :x-label "x"
                :y-label "y"
                :grid true
                :x-range [-5 5]
                :y-range [-125 125]}
        :datasets [{:title "x^2" :style "lines" :data squared-data}
        {:title "x^3" :style "lines" :data cubed-data}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

-- Generate data
local squared_data = {}
local cubed_data = {}
for i = -10, 10 do
local x = i * 0.5
table.insert(squared_data, {x, x*x})
table.insert(cubed_data, {x, x*x*x})
end

fplot.plot({
        options = {
                title = "Polynomials",
                ["x-label"] = "x",
                ["y-label"] = "y",
                grid = true,
                ["x-range"] = {-5, 5},
                ["y-range"] = {-125, 125}
        },
        datasets = {
                {title = "x^2", style = "lines", data = squared_data},
                {title = "x^3", style = "lines", data = cubed_data}
        }
})
```

Here, we've added a second dataset for $y = x^3$. We also added `:x-range` and `:y-range` to the options to set the viewing window of the plot. fplot automatically assigns different colors and line styles to each dataset.

3.5 Step 4: Saving to a File

If you want to save your plot instead of displaying it in a window, use the `:output-file` and `:output-type` options.

Fennel

```
(local fplot (require :fplot))

;; Generate data
(local squared-data [])
(local cubed-data [])
(for [x -5 5 0.5]
(table.insert squared-data [x (* x x)])
(table.insert cubed-data [x (* x x x)]))

(fplot.plot
{:options {:title "Polynomials"
                ;; ... other options from Step 3
                :x-range [-5 5]
                :y-range [-125 125]
                :output-file "polynomials.svg"
                :output-type "svg"}
        :datasets [{:title "x^2" :style "lines" :data squared-data}
        {:title "x^3" :style "lines" :data cubed-data}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

-- Generate data
local squared_data = {}
local cubed_data = {}
for i = -10, 10 do
local x = i * 0.5
table.insert(squared_data, {x, x*x})
table.insert(cubed_data, {x, x*x*x})
end

fplot.plot({
        options = {
                title = "Polynomials",
                ["output-file"] = "polynomials.svg",
                ["output-type"] = "svg",
                ["x-label"] = "x",
                ["y-label"] = "y",
                grid = false,
                ["x-range"] = {-5, 5},
                ["y-range"] = {-125, 125}
        },
```

```
        datasets = {
                {title = "x^2", style = "lines", data = squared_data},
                {title = "x^3", style = "lines", data = cubed_data}
        }
})
```

This will create an SVG file named `polynomials.svg` in the same directory. Common output types include `pngcairo`, `svg`, `pdfcairo`, and `jpeg`.

You now know the fundamentals of using fplot! To learn about all the available customization options, head over to the **Configuration Options** and **Dataset Configuration** sections.

4 Configuration Options

The `:options` table in `fplot` controls the overall appearance and properties of your plot. It's where you set titles, labels, ranges, output files, and more.

Below is a comprehensive list of all available options. Remember that when using Lua, keys with hyphens (like `:output-file`) must be written using the `["key-name"] = value` syntax.

4.1 General

Key	Type	Description	Example (Fennel)
`:title`	string	The main title of the plot.	`:title "My Awesome Plot"`
`:output-type`	string	The gnuplot terminal to use. Determines the output format. Common values: `wxt` (default, interactive window), `qt`, `pngcairo`, `svg`, `pdfcairo`.	`:output-type "pngcairo"`
`:output-file`	string	The path to save the output file. If `nil`, the plot is displayed in an interactive window.	`:output-file "my-plot.png"`
`:size`	table	A table `[width, height]` specifying the dimensions of the output in pixels.	`:size [800 600]`
`:font`	string	The font to use for text elements (e.g., "Arial", "Helvetica").	`:font "Arial"`
`:font-size`	number	The size of the font.	`:font-size 12`
`:gnuplot-executable`	string	The path to the gnuplot executable. Defaults to `"gnuplot"`, relying on the system `PATH`.	`:gnuplot-executable "/opt/local/bin/gnuplot"`

4.2 Axes and Labels

4.3 Tics

4.4 Appearance

4.5 Advanced

For options specific to individual datasets (like line color and style), see the **Dataset Configuration** section.

Key	Type	Description	Example (Fennel)
`:x-label`	string	The label for the x-axis.	`:x-label "Time (s)"`
`:y-label`	string	The label for the y-axis.	`:y-label "Velocity (m/s)"`
`:z-label`	string	The label for the z-axis (for 3D plots).	`:z-label "Height (m)"`
`:x-range`	table	A table `[min, max]` for the x-axis range. Use `"*"` for auto-scaling one end.	`:x-range [-10 10]`
`:y-range`	table	A table `[min, max]` for the y-axis range.	`:y-range [0 "*"]`
`:z-range`	table	A table `[min, max]` for the z-axis range.	`:z-range [-2 2]`
`:log-scale`	string	A string specifying which axes should use a logarithmic scale (e.g., `"x"`, `"xy"`, `"z"`).	`:log-scale "y"`

Key	Type	Description	Example (Fennel)
`:tics`	table	A map to control global tic settings for axes.	`:tics {:xtics "mirror", :ytics "rotate by 90"}`
`:xtics`	table	Sets custom tic marks for the x-axis. Can be a simple list of values or a list of `[label, value]` pairs.	`:xtics [["Low" 0] ["Mid" 50] ["High" 100]]`
`:ytics`	table	Sets custom tic marks for the y-axis.	`:ytics [0 10 20 30]`

Key	Type	Description	Example (Fennel)
`:grid`	boolean	If `true`, displays a grid on the plot.	`:grid true`
`:border`	boolean	If `true` (default), displays a border around the plot.	`:border false`
`:key`	table	Controls the plot's key (legend). Set `:enabled false` to hide it.	`:key {:enabled true :position "bottom left"}`
`:palette`	string	Sets the color palette for 3D plots or color-mapped data.	`:palette "defined (0 'blue', 1 'green', 2 'red')"`

Key	Type	Description	Example (Fennel)
`:labels`	list	A list of tables `[text, x, y, options]` to place text labels on the plot.	`:labels [["Peak" 1.5 9.8 "center font 'Verdana,10'"]]`
`:arrows`	list	A list of tables `[x1, y1, x2, y2, options]` to draw arrows.	`:arrows [[0 0 1.5 9.8 "head filled"]]`
`:objects`	list	A list of tables `[type, definition]` to draw geometric shapes.	`:objects [["rectangle" "from 0,0 to 1,1 fc rgb 'blue'"]]`
`:multiplot`	table	A table to configure a multiplot layout.	`:multiplot {:layout [2 2] :title "My Multiplot"}`
`:extra-opts`	list	A list of raw gnuplot commands to be inserted into the script.	`:extra-opts ["set style data histograms" "set style fill solid"]`

5 Dataset Configuration

The `:datasets` key in your main configuration table holds a list of tables. Each of these tables defines a single dataset to be drawn on your plot.

Here is a guide to the options available for configuring an individual dataset.

5.1 Core Dataset Properties

Key	Type	Description	Example (Fennel)
`:data`	table or string	The data for the plot. Can be a list of values (y-axis), a list of `[x, y]` pairs, or a filename string.	`:data [1 4 9 16]` or `:data "my-data.dat"`
`:title`	string	The title of the dataset, which appears in the plot's key (legend).	`:title "Temperature"`
`:style`	string	The plotting style. Common values: `lines`, `points`, `linespoints`, `dots`, `impulses`, `boxes`, `errorbars`.	`:style "linespoints"`
`:using`	string	Specifies which columns to use from the data. For example, `"1:3"` plots column 1 vs. column 3.	`:using "1:3:4" with yerrorbars`
`:axes`	string	The axes to plot this dataset against (e.g., `x1y1`, `x1y2`, `x2y2`). Useful for plots with multiple y-axes.	`:axes "x1y2"`
`:type`	string	The type of data source. Defaults to `"inline"`. Can be set to `"file"` to explicitly indicate the `:data` field is a path.	`:type "file"`

5.2 Styling Properties

These options override the default styles for a specific dataset.

Key	Type	Description	Example (Fennel)
`:color`	string	The color of the line or points. Can be a named color or a hex code.	`:color "#ff0000"` or `:color "blue"`
`:linetype`	number	The gnuplot linetype identifier.	`:linetype 2`
`:pointtype`	number	The gnuplot pointtype identifier.	`:pointtype 7`
`:fillstyle`	string	The fill style for filled shapes like boxes or histograms.	`:fillstyle "solid 0.5 border"`

5.3 Data Formats

`fplot` is flexible in how you provide your data.

1. Simple Y-Values (List)

A simple list of numbers will be plotted against their indices (1, 2, 3, ...).

```
:data [10 12 15 11 9]
```

2. X-Y Pairs (List of Lists/Tables)

This is the most common format for 2D plots.

```
:data [[0 10] [1 12] [2 15] [3 11] [4 9]]
```

3. Columnar Data (Transposed Table)

If it's more convenient to provide data as columns, fplot will automatically transpose it. This is detected when the number of inner tables (columns) is less than the number of items in the first inner table.

```
;; This is treated the same as the X-Y pairs above
:data [[0 1 2 3 4]      ;-- x-values
[10 12 15 11 9]]  ;-- y-values
```

4. 3D Grid Data (for `splot`)

For surface plots (e.g., with `pm3d`), provide a list of rows, where each row is a list of `[x y z]` points. `fplot` formats this correctly by adding blank lines between rows for gnuplot.

```
:data [;; Row 1
[[0 0 5] [0 1 6] [0 2 7]]
;; Row 2
[[1 0 8] [1 1 9] [1 2 8]]
;; Row 3
[[2 0 7] [2 1 6] [2 2 5]]]
```

5. Data from a File

To plot data directly from a file, provide the filename as a string in the `:data` field.

```
:data "path/to/your/data.dat"
```

The data file should be plain text, with columns separated by whitespace. For example `data.dat`:

```
# X Y
0  10
1  12
2  15
3  11
4  9
```

6 Advanced Features

This page covers some of the more advanced capabilities of `fplot`, such as creating composite plots, adding annotations, and using raw Gnuplot commands.

> **NOTE:** Fennel Macros are not included in this wiki page, because they have their own dedicated section.

6.1 Multiplot

`fplot` makes it easy to combine several plots into a single image using Gnuplot's multiplot feature. You enable this by passing multiple plot configuration tables to the `fplot.plot` function. The **last table** argument is special: it must contain the `:multiplot` configuration within its `:options` table.

The `:multiplot` table accepts the following keys:
`:layout`: A table `[rows, cols]` defining the grid for the subplots.
`:title`: An optional overall title for the entire multiplot figure.

Example: 2x1 Multiplot

This example creates two plots stacked vertically.

Fennel

```
(local fplot (require :fplot))
;; --- Data ---
(local sin-data [])
(local cos-data [])
(for [i 0 100]
(let [x (* i 0.1)]
(table.insert sin-data [x (math.sin x)])
(table.insert cos-data [x (math.cos x)])))
;; --- Plotting ---
(fplot.plot
;; Plot 1: Sine
{:options {:title "Sine"}
        :datasets [{:data sin-data}]}
;; Plot 2: Cosine
{:options {:title "Cosine"}
        :datasets [{:data cos-data}]}
;; Multiplot Configuration (in the last table)
{:options {:multiplot {:layout [2 1]
                       :title "Trigonometric Functions"}}})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

-- --- Data ---
local sin_data = {}
local cos_data = {}
for i = 0, 100 do
local x = i * 0.1
table.insert(sin_data, {x, math.sin(x)})
table.insert(cos_data, {x, math.cos(x)})
end
```

```
-- --- Plotting ---
fplot.plot(
-- Plot 1: Sine
{
        options = { title = "Sine" },
        datasets = { { data = sin_data } }
},

-- Plot 2: Cosine
{
        options = { title = "Cosine" },
        datasets = { { data = cos_data } }
},

-- Multiplot Configuration (in the last table)
{
        options = {
                multiplot = {
                        layout = {2, 1},
                        title = "Trigonometric Functions"
                }
        }
}
)
```

(The resulting plot generated by fplot)

[Image: P6.ex.Plot1]

6.2 Labels, Arrows, and Objects

You can add arbitrary annotations to your plot using the `:labels`, `:arrows`, and `:objects` options.

- `:labels`: Add text to the plot. Each label is a table `[text, x, y, options]`.
- `:arrows`: Draw arrows. Each arrow is a table `[x1, y1, x2, y2, options]`.
- `:objects`: Draw shapes (rectangles, circles, etc.). Each object is a table `[type, definition]`.

Example: Annotating a Peak

Fennel

```
(fplot.plot
{:options {:title "Annotated Plot"
                :x-range [0 10]
                :y-range [0 12]
                :labels [["Important Peak" 5 10.5 "center"]]
                :arrows [[7 8 5.2 9.8 "head filled"]]
                :objects [["rectangle" "from 4,0 to 6,10 fs transparent solid
                   0.1 noborder"]]}
        :datasets [{:data [[1 2] [3 7] [5 10] [7 6] [9 3]]
                :style "linespoints"}]})
```

Lua

```lua
fplot.plot({
        options = {
                title = "Annotated Plot",
                ["x-range"] = {0, 10},
                ["y-range"] = {0, 12},
                labels = {
                        {"Important Peak", 5, 10.5, "center"}
                },
                arrows = {
                        {7, 8, 5.2, 9.8, "head filled"}
                },
                objects = {
                        {"rectangle", "from 4,0 to 6,10 fs transparent solid 0.1
                            noborder"}
                }
        },
        datasets = {
                {
                        data = {{1, 2}, {3, 7}, {5, 10}, {7, 6}, {9, 3}},
                        style = "linespoints"
                }
        }
})
```

(The resulting plot generated by fplot)

[Image: P6.ex.Plot2]

6.3 Custom Tics

You can take full control over the axis tic marks using the `:xtics` and `:ytics` options. This is useful for categorical data or for highlighting specific points. Provide a list of `["label", value]` pairs.

Example: Labeled Tics

Fennel

```fennel
(local fplot (require :fplot))

(fplot.plot
{:options {:title "Quarterly Sales"
                :xtics [["Q1" 1] ["Q2" 2] ["Q3" 3] ["Q4" 4]]
                :ytics [0 10000 20000 30000]
                :y-label "Revenue ($)"}
        :datasets [{:data [[1 15000] [2 22000] [3 18000] [4 25000]]
                :style "boxes"
                :title "Sales"}]})
```

Lua

```lua
require("fennel").install()
local fplot = require("fplot")

fplot.plot({
        options = {
```

```
                title = "Quarterly Sales",
                xtics = {{"Q1", 1}, {"Q2", 2}, {"Q3", 3}, {"Q4", 4}},
                ytics = {0, 10000, 20000, 30000},
                ["y-label"] = "Revenue ($)"
        },
        datasets = {
                {
                        data = {{1, 15000}, {2, 22000}, {3, 18000}, {4, 25000}},
                        style = "boxes",
                        title = "Sales"
                }
        }
})
```

(The resulting plot generated by fplot)

[Image: P6.ex.Plot3]

6.4 Extra Options (:extra-opts)

For features that are not directly exposed by `fplot`, you can use the `:extra-opts` option to pass raw command strings directly to the Gnuplot script. This provides an escape hatch for accessing the full power of Gnuplot.

Example: Setting a Custom Fill Style

Fennel

```
(local fplot (require :fplot))

(fplot.plot
{:options {:title "Histogram"
                :extra-opts ["set style fill solid 0.6 border -1"
                "set style histogram clustered gap 2"
                "set style data histograms"]}
        :datasets [{:title "Group A" :data [10 15 12]}
        {:title "Group B" :data [12 11 14]}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

fplot.plot({
        options = {
                title = "Histogram",
                ["extra-opts"] = {
                        "set style fill solid 0.6 border -1",
                        "set style histogram clustered gap 2",
                        "set style data histograms"
                }
        },
        datasets = {
                { title = "Group A", data = {10, 15, 12} },
                { title = "Group B", data = {12, 11, 14} }
        }
})
```

(The resulting plot generated by fplot)

[Image: P6.ex.Plot4]

---

7 Using Macros and Reusable Components with Fplot

A key advantage of using a programmatic plotting library like `fplot` is the ability to create reusable, composable components. This page explains the difference between Gnuplot's native macros and the more flexible approach of using Fennel (or Lua) to manage your plotting.

7.1 Fennel Macros vs. Lua Functions: A Quick Clarification

It's important to clarify the term "macro."

- **Fennel**, as a Lisp, has a **true macro system**. This means Fennel macros are code that writes code, running at **compile-time** to transform your program before it even becomes Lua. This allows you to create new syntax and fundamentally change how the language works for your specific needs.
- **Lua**, on the other hand, does not have a macro system. When we talk about creating "macros" in a Lua context for `fplot`, we are using the term informally to describe a powerful **runtime** pattern: **reusable functions that generate complex data structures** (like tables for options or datasets). We'll refer to these as "reusable components" or "factory functions" in the Lua examples.

7.2 Gnuplot's Native Macros

Gnuplot has a built-in macro system using the `@` character (e.g., `plot @my_macro`). While `fplot` does not have a feature to directly use these macros in the plot command, you can still **define** them using the `:extra-opts` table.

This is useful if you have existing Gnuplot scripts or need to set a simple string substitution.

```
(fplot.plot
{:options {:title "Plotting with a Gnuplot Macro"
                  :x-range [-5 5]
                  ;; Define the macro here
                  :extra-opts ["my_func = 'sin(x) / x'"]}
         ;; ...
})
```

7.3 Fennel: The Real Macro System

The true power of fplot comes from using Fennel itself to create reusable components. Because Fennel has a true macro system, you can create very powerful abstractions. But for most plotting scenerios, simple functions that just return tables are usually more than enough.

1. Reusable Data Generators

You can write functions that generate complex datasets.

```
;; A function for generating sinc data
(fn generate-sinc-data [steps]
(let [data []
step-size (/ 40 steps)]
(for [i (- (/ steps 2)) (/ steps 2)]
(let [x (* i step-size)
row []]
(for [j (- (/ steps 2)) (/ steps 2)]
```

```
(let [y (* j step-size)
r (+ (math.sqrt (+ (* x x) (* y y))) 1e-9)
z (/ (math.sin r) r)]
(table.insert row [x y z])))
(table.insert data row)))
data))

;; Use the data generator
(local sinc-data (generate-sinc-data 40))
(fplot.splot {:options {:title "Sinc Function"}
        :datasets [{:data sinc-data :style "pm3d"}]})
```

2. Reusable Style Templates

Create functions that return tables with your preferred plot options. This allows you to maintain a consistent style across many different plots.

```
;; A function for a publication-quality plot style
(fn publication-style [title]
{:title title
        :font "Helvetica,14"
        :border true
        :grid true})

;; Use the style template by calling the function
(fplot.plot
{:options (publication-style "My Publication Plot")
        :datasets [...]})
```

3. Putting It All Together

The real magic happens when you combine these components. You can define a base style, a plot-specific style, and then merge them together. The example below is a complete, and runnable script. Feel free to try it!

Fennel

```
(local fplot (require :fplot))

;; Helper function to recursively clone a table.
(fn table-clone [tbl]
(let [copy {}]
(each [k v (pairs tbl)]
(tset copy k (if (= (type v) "table") (table-clone v) v)))
copy))

;; Helper function to merge tables.
(fn merge-tables [a b]
(let [result (table-clone a)]
(each [k v (pairs b)]
(if (and (= (type v) "table") (= (type (. result k)) "table"))
(tset result k (merge-tables (. result k) v))
(tset result k v)))
result))

;; --- Our Reusable Components ---
(fn generate-line-data [num-points]
```

```
(let [data []]
(for [i 1 num-points]
(table.insert data [i (math.random)]))
data))

(fn base-style []
{:font "Arial,12"
        :grid true
        :output-type "pngcairo"
        :size [800 600]})

(fn scatter-plot-style []
{:style "points"
        :pointtype 7
        :color "#3366cc"})

;; --- The Plotting Script ---
(local my-data (generate-line-data 100))

(local plot-specific-opts
{:title "My Composed Plot"
        :x-label "Index"
        :y-label "Random Value"
        :output-file "my-composed-plot.png"})

(local final-options (merge-tables (base-style) plot-specific-opts))
(local final-dataset (merge-tables (scatter-plot-style) {:data my-data}))

(fplot.plot {:options final-options
        :datasets [final-dataset]})

(print "Plot generated at my-new-composed-plot.png")
```

The Lua Equivalent: Reusable Functions

```
require("fennel").install()
local fplot = require("fplot")

-- Helper functions
local function table_clone(tbl)
local copy = {}
for k, v in pairs(tbl) do
if type(v) == "table" then
copy[k] = table_clone(v)
else
copy[k] = v
end
end
return copy
end

local function merge_tables(a, b)
local result = table_clone(a)
for k, v in pairs(b) do
if type(v) == "table" and type(result[k]) == "table" then
result[k] = merge_tables(result[k], v)
else
result[k] = v
```

```
end
end
return result
end

-- --- Our Reusable Components ---
local function generate_line_data(num_points)
local data = {}
for i = 1, num_points do
table.insert(data, {i, math.random()})
end
return data
end

local function base_style()
return {
        font = "Arial,12",
        grid = true,
        ["output-type"] = "pngcairo",
        size = {800, 600}
}
end

local function scatter_plot_style()
return {
        style = "points",
        pointtype = 7,
        color = "#3366cc"
}
end

-- --- The Plotting Script ---
local my_data = generate_line_data(100)

local plot_specific_opts = {
        title = "My Composed Plot",
        ["x-label"] = "Index",
        ["y-label"] = "Random Value",
        ["output-file"] = "my-composed-plot.png"
}

local final_options = merge_tables(base_style(), plot_specific_opts)
local final_dataset = merge_tables(scatter_plot_style(), {data = my_data})

fplot.plot({
        options = final_options,
        datasets = {final_dataset}
})

print("Plot generated at my-composed-plot.png")
```

---

8 Examples

This page provides a gallery of examples to showcase the kinds of plots you can create with `fplot`. Each example includes the code in both Fennel and Lua, along with the resulting plot.

8.1 1. Line and Point Plot

A standard plot combining smooth lines and distinct data points.

Fennel

```
(local fplot (require :fplot))

(local data1 [])
(for [i 0 50]
(let [x (* i 0.2)]
(table.insert data1 [x (* (math.exp (* -0.1 x)) (math.cos x))])))

(local data2 [[1 0.5] [3 0.1] [5 -0.2] [7 -0.1] [9 0.05]])

(fplot.plot
{:options {:title "Damped Oscillation with Data Points"
                :x-label "Time"
                :y-label "Amplitude"
                :grid true
                :key {:position "top right"}}
        :datasets [{:title "Model"
                :style "lines"
                :color "blue"
                :data data1}
        {:title "Measurements"
                :style "points"
                :color "red"
                :pointtype 7
                :data data2}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

local data1 = {}
for i = 0, 50 do
local x = i * 0.2
table.insert(data1, {x, math.exp(-0.1 * x) * math.cos(x)})
end

local data2 = {{1, 0.5}, {3, 0.1}, {5, -0.2}, {7, -0.1}, {9, 0.05}}

fplot.plot({
        options = {
                title = "Damped Oscillation with Data Points",
                x_label = "Time",
                y_label = "Amplitude",
                grid = true,
```

```
                key = {position = "top right"}
        },
        datasets = {
                {
                        title = "Model",
                        style = "lines",
                        color = "blue",
                        data = data1
                },
                {
                        title = "Measurements",
                        style = "points",
                        color = "red",
                        pointtype = 7,
                        data = data2
                }
        }
})
```

(The resulting plot generated by fplot)

[Image: P8.ex.Plot1]

8.2 2. Bar Chart / Histogram

Creating a bar chart using the `boxes` style, which is great for categorical data.

Fennel

```
(local fplot (require :fplot))

(fplot.plot
{:options {:title "Annual Production by Factory"
                :y-label "Units Produced"
                :x-range [0.5 4.5]
                :y-range [0 50000]
                :xtics [["North" 1] ["South" 2] ["East" 3] ["West" 4]]
                :extra-opts ["set style fill solid 0.8"]}
        :datasets [{:title "Production"
                :style "boxes"
                :data [[1 42000] [2 35000] [3 48000] [4 29000]]}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

fplot.plot({
        options = {
                title = "Annual Production by Factory",
                y_label = "Units Produced",
                x_range = {0.5, 4.5},
                y_range = {0, 50000},
```

```
                xtics = {{"North", 1}, {"South", 2}, {"East", 3}, {"West", 4}},
                extra_opts = {"set style fill solid 0.8"}
        },
        datasets = {
                {
                        title = "Production",
                        style = "boxes",
                        data = {{1, 42000}, {2, 35000}, {3, 48000}, {4, 29000}}
                }
        }
})
```

(The resulting plot generated by fplot)

[Image: P8.ex.Plot2]

8.3 3. 3D Surface Plot

Using `fplot.splot` to visualize three-dimensional data.

Fennel

```
(local fplot (require :fplot))

(local data [])
(for [i -20 20]
(let [x (* i 0.5)
row []]
(for [j -20 20]
(let [y (* j 0.5)
r (+ (math.sqrt (+ (* x x) (* y y))) 1e-6)
z (/ (math.sin r) r)]
(table.insert row [x y z])))
(table.insert data row)))

(fplot.splot
{:options {:title "Sinc Function"
                :z-label "z"
                :palette "viridis"}
        :datasets [{:data data
                :style "pm3d"
                :title ""}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

local data = {}
for i = -20, 20 do
local x = i * 0.5
local row = {}
for j = -20, 20 do
```

```
local y = j * 0.5
local r = math.sqrt(x*x + y*y) + 1e-6
local z = math.sin(r) / r
table.insert(row, {x, y, z})
end
table.insert(data, row)
end

fplot.splot({
        options = {
                title = "Sinc Function",
                z_label = "z",
                palette = "viridis"
        },
        datasets = {
                {
                        data = data,
                        style = "pm3d",
                        title = ""
                }
        }
})
```

(The resulting plot generated by fplot)

[Image: P8.ex.Plot3]

8.4 4. Logarithmic Scale Plot

Useful for visualizing data that spans several orders of magnitude.

Fennel

```
(local fplot (require :fplot))

(local data [])
(for [x 1 100]
(table.insert data [x (* 1000 (math.exp (* -0.1 x)))]))

(fplot.plot
{:options {:title "Exponential Decay"
                :x-label "Time"
                :y-label "Value"
                :log-scale "y"
                :grid true}
        :datasets [{:title "Decay"
                :style "lines"
                :data data}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

local data = {}
for x = 1, 100 do
table.insert(data, {x, 1000 * math.exp(-0.1 * x)})
end

fplot.plot({
        options = {
                title = "Exponential Decay",
                x_label = "Time",
                y_label = "Value",
                log_scale = "y",
                grid = true
        },
        datasets = {
                {
                        title = "Decay",
                        style = "lines",
                        data = data
                }
        }
})
```

(The resulting plot generated by fplot)

[Image: P8.ex.Plot4]

---

9 Biotech Applications

This page demonstrates how to generate plots commonly used in Biotech.

9.1 1. Volcano Plot for Gene Expression

Volcano plots are used to visualize differential gene expression results of RNA-Seq and/or microarray experiments. They plot statistical significance (-log10 p-value) against magnitude of change (log2 fold change).

This example shows how to create a volcano plot, highlighting significantly up- and down-regulated genes.

Fennel

```
(local fplot (require :fplot))

;; --- Mock Data Generation ---
(math.randomseed 42)
(local up-regulated [])
(local down-regulated [])
(local non-significant [])

(for [_ 1 500]
(let [log2fc (- -4 (* (math.random) 4)) ; range [-8, -4]
p-value (* (math.random) 1e-6)]
(table.insert down-regulated [log2fc (* -1 (math.log10 p-value))])))

(for [_ 1 500]
(let [log2fc (+ 4 (* (math.random) 4)) ; range [4, 8]
p-value (* (math.random) 1e-6)]
(table.insert up-regulated [log2fc (* -1 (math.log10 p-value))])))

(for [_ 1 9000]
(let [log2fc (* (- (math.random) 0.5) 8) ; range [-4, 4]
p-value (math.random)]
(table.insert non-significant [log2fc (* -1 (math.log10 p-value))])))


;; --- Plotting ---
(fplot.plot
{:options {:title "Volcano Plot of Differential Gene Expression"
                :x-label "log2(Fold Change)"
                :y-label "-log10(p-value)"
                :grid true
                :key {:enabled false}}
        :datasets [{:style "points" :pointtype 7 :color "#cccccc" :data
           non-significant}
        {:style "points" :pointtype 7 :color "#ff4136" :data up-regulated}
        {:style "points" :pointtype 7 :color "#0074d9" :data down-regulated}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")
```

```
-- --- Mock Data Generation ---
math.randomseed(42)
local up_regulated = {}
local down_regulated = {}
local non_significant = {}

for _ = 1, 500 do
local log2fc = -4 - math.random() * 4
local p_value = math.random() * 1e-6
table.insert(down_regulated, {log2fc, -math.log10(p_value)})
end

for _ = 1, 500 do
local log2fc = 4 + math.random() * 4
local p_value = math.random() * 1e-6
table.insert(up_regulated, {log2fc, -math.log10(p_value)})
end

for _ = 1, 9000 do
local log2fc = (math.random() - 0.5) * 8
local p_value = math.random()
table.insert(non_significant, {log2fc, -math.log10(p_value)})
end

-- --- Plotting ---
fplot.plot({
        options = {
                title = "Volcano Plot of Differential Gene Expression",
                x_label = "log2(Fold Change)",
                y_label = "-log10(p-value)",
                grid = true,
                key = {enabled = false}
        },
        datasets = {
                {style = "points", pointtype = 7, color = "#cccccc", data =
                   non_significant},
                {style = "points", pointtype = 7, color = "#ff4136", data =
                   up_regulated},
                {style = "points", pointtype = 7, color = "#0074d9", data =
                   down_regulated}
        }
})
```

(The resulting plot generated by fplot)

[Image: P9.ex.Plot1]

9.2 2. Protein Thermal Melt Curve

A thermal melt curve from a Differential Scanning Fluorimetry (DSF) experiment is used to assess protein stability. This plot shows fluorescence vs. temperature, and the inflection point of the curve is the melting temperature (Tm). This example plots two curves, representing a protein with and without a stabilizing ligand.

Fennel

```
(local fplot (require :fplot))

;; --- Mock Data Generation ---
(fn sigmoid [x x0 k]
(/ 1 (+ 1 (math.exp (- (* k (- x x0)))))))

(local protein-only [])
(local with-ligand [])
(for [temp 25 95 1]
(let [base-fluor (* (math.random) 0.1)
tm1 55
tm2 65]
(table.insert protein-only [temp (+ 10 base-fluor (* 80 (sigmoid temp tm1 0.5)))
   ])
(table.insert with-ligand [temp (+ 10 base-fluor (* 80 (sigmoid temp tm2 0.5)))
   ])))

;; --- Plotting ---
(fplot.plot
{:options {:title "Protein Thermal Shift Assay"
                :x-label "Temperature (°C)"
                :y-label "Relative Fluorescence Units (RFU)"
                :key {:position "bottom right"}}
        :datasets [{:title "Apo Protein"
                :style "lines"
                :color "black"
                :data protein-only}
        {:title "Protein + Ligand"
                :style "lines"
                :color "blue"
                :data with-ligand}]})
```

Lua

```
require("fennel").install()
local fplot = require("fplot")

-- --- Mock Data Generation ---
local function sigmoid(x, x0, k)
return 1 / (1 + math.exp(-k * (x - x0)))
end

local protein_only = {}
local with_ligand = {}
for temp = 25, 95, 1 do
local base_fluor = math.random() * 0.1
local tm1 = 55
local tm2 = 65
table.insert(protein_only, {temp, 10 + base_fluor + 80 * sigmoid(temp, tm1, 0.5)
   })
table.insert(with_ligand, {temp, 10 + base_fluor + 80 * sigmoid(temp, tm2, 0.5)
   })
end
```

```
-- --- Plotting ---
fplot.plot({
        options = {
                title = "Protein Thermal Shift Assay",
                x_label = "Temperature (°C)",
                y_label = "Relative Fluorescence Units (RFU)",
                key = {position = "bottom right"}
        },
        datasets = {
                {
                        title = "Apo Protein",
                        style = "lines",
                        color = "black",
                        data = protein_only
                },
                {
                        title = "Protein + Ligand",
                        style = "lines",
                        color = "blue",
                        data = with_ligand
                }
        }
})
```

(The resulting plot generated by fplot)

[Image: P9.ex.Plot2]

---

10 Troubleshooting Guide

This guide provides solutions to common problems you might encounter when using `fplot` in different environments.

10.1 General Issues

Problem: `gnuplot` not found or command fails.

When you run an `fplot` script, and you see an error like `sh: gnuplot: command not found` or a non-zero exit code.

Solution:

1. **Verify `gnuplot` Installation:** Make sure `gnuplot` is installed on your system. See the Installation guide.
2. **Check System `PATH`:** The `gnuplot` executable must be in your system's `PATH`. You can check this by opening a terminal and typing `gnuplot --version`. If it's not found, you need to add its location to your `PATH` environment variable.
3. **Hardcode the Path:** As a last resort, you can edit your local `fplot.fnl` (or `fplot.lua`) file and change the `gnuplot-executable` variable to the full, absolute path of the `gnuplot` executable on your system.

Problem: The plot window appears and immediately closes.

Solution:

This typically happens when gnuplot is not run in "persistent" mode. fplot automatically handles this by adding the -p flag or a pause mouse close command when it's not saving to a file. If you are running gnuplot scripts manually, make sure to use the -p flag: `gnuplot -p your_script.gp`.

10.2 MSYS2 (on Windows)

The MSYS2 environment can be tricky because of how it handles file paths.

Problem: `fplot` fails with errors like "Can't open script file" or "Cannot find data file".

This is the most common issue on MSYS2 and is caused by **path mangling**. MSYS2 automatically converts POSIX-style paths (e.g., `/c/Users/Me`) to Windows-style paths (`C:\Users\Me`) when calling native Windows programs like `gnuplot`. This can corrupt the paths to the temporary script and data files that `fplot` creates.

Solution 1: Disable Path Mangling (Recommended)

You can tell MSYS2 not to convert paths by setting an environment variable before running your script.

```
# In your MSYS2 terminal
export MSYS2_ARG_CONV_EXCL="*"
fennel your_plot_script.fnl
```

To make this permanent, add `export MSYS2_ARG_CONV_EXCL="*"` to your `~/.bashrc` or `~/.bash_profile` file in your MSYS2 home directory.

Solution 2: Use the MinGW `gnuplot` Package

Within your MSYS2 environment, it's best to use the `gnuplot` package from the MinGW repositories, as it's a native Windows build that integrates well with the environment. You can install it using `pacman`:

```
pacman -S mingw-w64-x86_64-gnuplot
```

This version is generally more reliable for GUI applications than a purely POSIX-emulated build. However, even with this version, the path mangling described in Solution 1 is often still a problem, so disabling it is the most robust fix.

10.3 WSL2 (Windows Subsystem for Linux)

WSL2 runs a full Linux kernel. On modern Windows 10 and 11, the **WSLg** feature provides built-in support for Linux GUI applications, so they should work out of the box.

Problem: The script runs, but no plot window appears. You might see an error like "cannot open display" or "Gnuplot terminal type set to 'unknown'".

Solution for Modern Windows (with WSLg):

1. **Ensure WSL is Up-to-Date:** Open a Windows PowerShell or Command Prompt and run `wsl --update`. This will ensure you have the latest version with WSLg.
2. **Install a GUI-enabled `gnuplot`:** Your Linux distribution needs a version of `gnuplot` that can create a graphical window.

   ```
   # On Debian/Ubuntu
   sudo apt-get update
   sudo apt-get install gnuplot-x11
   ```

   This should be all you need. When you run your `fplot` script from WSL2, the plot window should appear on your Windows desktop automatically(appearance is slow).

Solution for Older Windows Versions (without WSLg):

If you are on an older build of Windows that doesn't support WSLg, you will need to manually install an X server on Windows.

1. **Install an X Server on Windows:** Download and install a free X server like VcXsrv on your Windows machine.
2. **Launch the X Server:** Launch VcXsrv and choose "Multiple windows", "Display number -1", and on the final page, check "Disable access control".
3. **Configure the `DISPLAY` Variable in WSL2:** Your Linux environment needs to know where the display is. Add the following line to your `~/.bashrc` file in your WSL2 home directory:

   ```
   export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print $2;
       exit;}'):0.0
   ```

4. **Reload Your Shell:** Close and reopen your WSL2 terminal, or run `source ~/.bashrc`. After this, GUI applications from WSL2 should appear on your desktop.

10.4 Native Linux / macOS

`fplot` is most at home on these systems and issues are rare.

Problem: `fplot` complains about a missing terminal type (e.g., "Terminal type set to 'unknown'").

Solution:
Your gnuplot installation may be missing support for interactive terminals. When you installed gnuplot, you may have gotten a "core" or "nogui" version. Re-install it and ensure you get a version with qt or x11 support.

- **Debian/Ubuntu:** `sudo apt-get install gnuplot-x11` or `sudo apt-get install gnuplot-qt`
- **Arch Linux:** The default `gnuplot` package should be sufficient.
- **macOS (Homebrew):** `brew install gnuplot` should install a version with the `qt` terminal.

You can check which terminals your `gnuplot` supports by running `gnuplot -e "show terminal"`.

10.5 A Note on Gnuplot Terminals (GUI Backends)

Problem: A plot window appears, but it's buggy, slow, or doesn't appear at all, even though `gnuplot` is installed correctly.

Solution:
gnuplot uses different internal engines, called "terminals," to draw its interactive plot windows. The most common ones are wxt (the default on many systems), qt, and x11. Sometimes, the default terminal may have issues with your specific desktop environment or system configuration, especially in complex setups like MSYS2.

If you suspect this is the issue, you can force `fplot` to use a different terminal by setting the `:output-type` option in your plot configuration.

Example (forcing the `qt` terminal):

```
(fplot.plot
{:options {:title "My Plot"
                  :output-type "qt"} ; <-- Force the qt terminal
          :datasets [;...
          ]})
```

This is a powerful troubleshooting step. If the default `wxt` terminal fails, try `qt`. If `qt` fails, try `x11` (on Linux). Make sure you have the necessary support libraries installed for the terminal you want to use (e.g., the `gnuplot-qt` package on Debian/Ubuntu for the `qt` terminal).